

자유스터디 활동보고서(6차)

※ 빨간색 글씨를 참고한 뒤 삭제 후 작성 바랍니다.

팀명	KHUDA	학습 주제	딥러닝과 자연어처리	
교과목명/ 교수자명	딥러닝과 자연어처리			
학습 일시	2025년 5월 25일 오후 2시 ~ 2025년 5월 25일 오후 4시		학습 시간	총 2시간
진행 방식 (소통 도구)	Google meet			
참석자	성명		인원	
	김재욱, 김민서, 오종현, 한지훈, 조민우		총 5명	
회차별 학습 주제	Building a GPT			
세부 학습 내용 (3가지 이상)	학습 내용 1. 텍스트 기반 언어 모델 학습을 위한 전처리 과정 실습 GPT 모델의 학습을 위해 “tinysakespeare” 데이터셋을 활용하여 문자 단위로 데이터를 분리하고, 각 문자에 정수 인덱스를 매핑하는 전처리 과정을 실습하였다. stoi, itos와 같은 구조를 직접 구현하며 실제 텍스트 데이터에 대한 토큰나이징 및 임베딩의 개념을 이해할 수 있었다.			
	2. 입력 시퀀스와 배치 생성 등 GPT 학습 흐름 구성 방식 학습 언어 모델 학습에서 입력 시퀀스를 고정 길이로 나누고, 이를 배치 단위로 구성해보는 과정을 실습하며, 슬라이딩 윈도우 형태의 시퀀스 생성을 통해 GPT 모델이 데이터를 어떻게 학습하는지 이해해보았다.			
	3. 추가 모델 확장에 필요한 개념, 이론 학습 이번 실습을 통해 구성한 기본적인 GPT 학습 파이프라인을 바탕으로, 향후 모델 성능을 높이기 위해 어떤 개념들을 추가로 적용할 수 있을지에 대해 학습하였다. Self-Attention의 확장 형태인 Multi-head Attention, 위치 정보를 효과적으로 반영하기 위한 Positional Encoding, 모델 일반화를 위한 Regularization 기법 등 다양한 개선 요소들이 모델의 품질에 영향을 미칠 수 있음을 학습했다.			
회차별 팀원 역할 인당 1줄	김재욱 : 6회차 전체 실습 코드 설명 및 진행 조민우 : 6회차 스터디 과제 공지 및 안내 한지훈 : 지난 회차 스터디 과제 Review 김민서 : 추가적인 모델 확장을 위한 최신 기술 논문 소개 오종현 : 6회차 스터디 시간 조율 및 공지, 활동보고서 작성			

학습 활동
증빙자료

학습
자료

```
gpt-devipynb
파일 수정 보기 삽입 런타임 도구 도움말

# We always start with a dataset to train on. Let's download the tiny shakespeare dataset
!wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

--2023-01-17 01:39:27-- https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1115394 (1.1M) [text/plain]
Saving to: 'input.txt'

input.txt      100%[=====] 1.06M --.-KB/s  in 0.04s

2023-01-17 01:39:28 (29.0 MB/s) - 'input.txt' saved [1115394/1115394]

[] # read it in to inspect it
with open('input.txt', 'r', encoding='utf-8') as f:
    text = f.read()

[] print("length of dataset in characters: ", len(text))

length of dataset in characters: 1115394

[] # let's look at the first 1000 characters
print(text[:1000])

First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
```

```
gpt-devipynb
파일 수정 보기 삽입 런타임 도구 도움말

Notes:

• Attention is a communication mechanism. Can be seen as nodes in a directed graph looking at each other and aggregating information with a weighted sum from all nodes that point to them, with data-dependent weights.
• There is no notion of space. Attention simply acts over a set of vectors. This is why we need to positionally encode tokens.
• Each example across batch dimension is of course processed completely independently and never 'talk' to each other
• In an 'encoder' attention block just delete the single line that does masking with tril, allowing all tokens to communicate. This block here is called a 'decoder' attention block because it has triangular masking, and is usually used in autoregressive settings, like language modeling.
• 'self-attention' just means that the keys and values are produced from the same source as queries. In 'cross-attention', the queries still get produced from x, but the keys and values come from some other, external source (e.g. an encoder module)
• 'Scaled' attention additionally divides wei by 1/sqrt(head_size). This makes it so when input Q/K are unit variance, wei will be unit variance too and Softmax will stay diffuse and not saturate too much. Illustration below

[] k = torch.randn(8,T,head_size)
q = torch.randn(8,T,head_size)
wei = q @ k.transpose(-2, -1) * head_size**0.5

[] k.var()

tensor(1.0449)

[] q.var()

tensor(1.0700)

[] wei.var()
```

```
gpt-devipynb
파일 수정 보기 삽입 런타임 도구 도움말

[] # Let's now split up the data into train and validation sets
n = int(0.9*len(data)) # first 90% will be train, rest val
train_data = data[:n]
val_data = data[n:]

block_size = 8
train_data[block_size:]

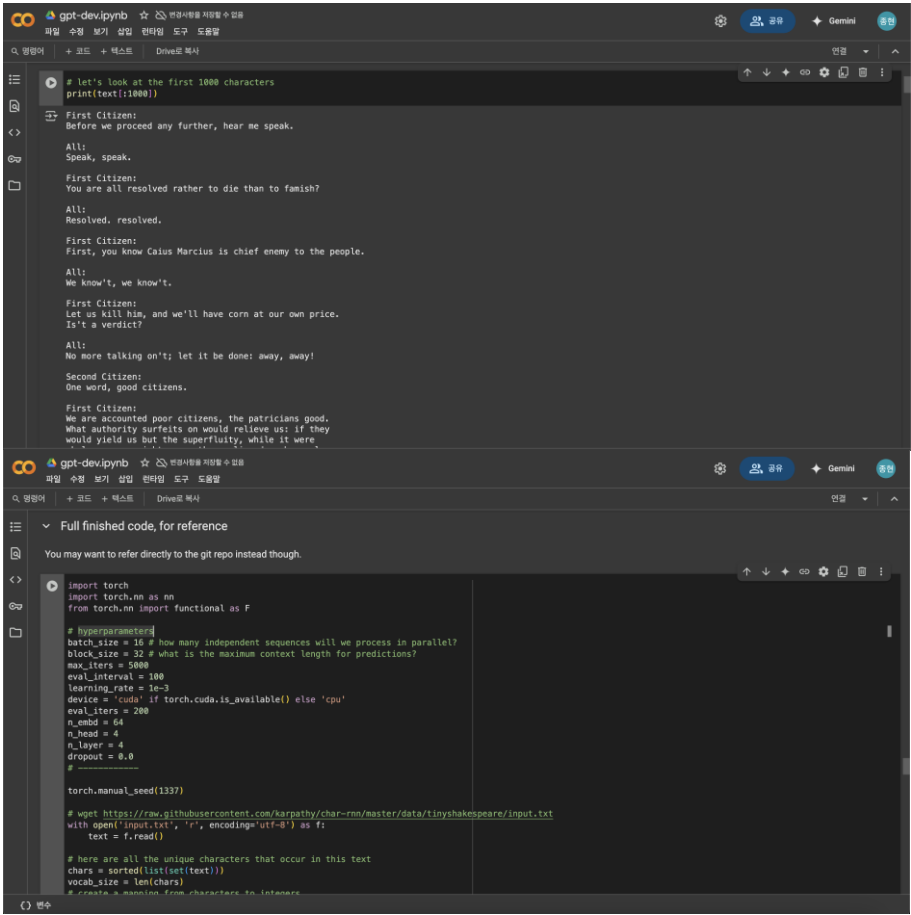
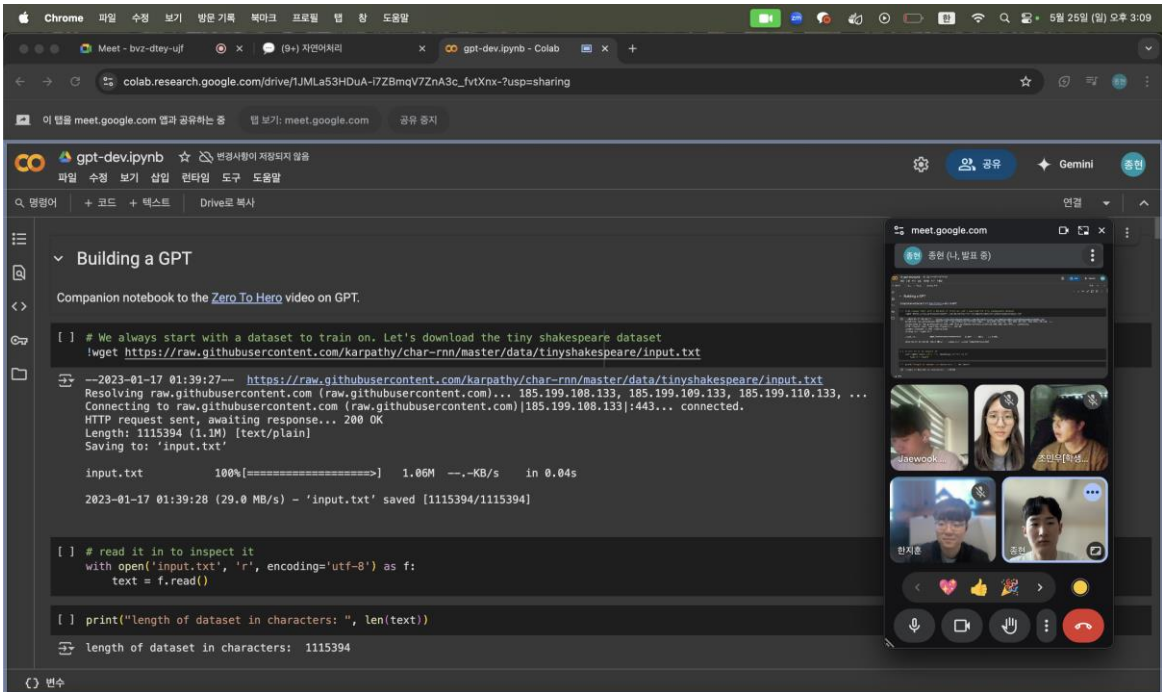
tensor([18, 47, 56, 57, 58, 1, 15, 47, 58])

[] x = train_data[block_size:]
y = train_data[block_size:]
for t in range(block_size):
    context = x[:t+1]
    target = y[t]
    print(f"when input is {context} the target: {target}")

when input is tensor([18]) the target: 47
when input is tensor([18, 47]) the target: 56
when input is tensor([18, 47, 56]) the target: 57
when input is tensor([18, 47, 56, 57]) the target: 58
when input is tensor([18, 47, 56, 57, 58]) the target: 1
when input is tensor([18, 47, 56, 57, 58, 1]) the target: 15
when input is tensor([18, 47, 56, 57, 58, 1, 15]) the target: 47
when input is tensor([18, 47, 56, 57, 58, 1, 15, 47]) the target: 58

[] torch.manual_seed(1337)
batch_size = 4 # how many independent sequences will we process in parallel?
block_size = 8 # what is the maximum context length for predictions?

def get_batch(split):
```

	
활동 사진 (1장)	

학습 자료 첨부